Common Concepts

Module Outcome:

- Build expressions (`collection - budget`) and string/date helpers MySQL actually has
- CONCAT names, format dates, and date-diff a stay or a leave request
- Keep functions in SELECT / WHERE without inventing Oracle syntax

Note: This course uses **MySQL 8+** functions. Key MySQL functions: `CONCAT()` , `LENGTH()` , `DATE_FORMAT()` , `DATEDIFF()` .

SQL Expressions

We can write **expressions** in various SQL clauses. Expressions can comprise of various data types like integers, floats, strings, datetime etc.

Using Expressions in SELECT Clause

Example: Get profits of all movies. (Consider profit as difference between collection and budget.)

```
SELECT
    id, name, (collection_in_cr - budget_in_cr) AS profit
FROM
    movie;
```

Output:

id	name	profit
1	The matrix	40.31
2	Inception	67.68
3	The Dark knight	82.5
...	...	...

Note: In MySQL, use `CONCAT()` to join strings. (SQLite uses `||` instead.)

Example 2: Get the movie name and genre in the following format: movie_name - genre.

```
SELECT
    CONCAT(name, ' - ', genre) AS movie_genre
FROM
    movie;
```

Output:

movie_genre
The Matrix - Sci-fi
Inception - Action
The Dark knight - Drama
Toy Story 3 - Animation
...

Using Expressions in WHERE Clause

Example: Get all the movies with a profit at least 50 crores.

```
SELECT
    *
FROM
    movie
WHERE
    (collection_in_cr - budget_in_cr) >= 50;
```

Output:

id	name	genre	budget_in_cr	collection_in_cr	rating	release
2	Inception	Action	16.0	83.68	8.8	2010-07-14
3	The dark knight	Action	18.0	100.5	9.0	2008-07-16
4	Toy story 3	Animation	20.0	1006.7	8.5	2010-06-25
...	...	...	...	...	...	...

Using Expressions in UPDATE Clause

Example: Scale down rating from 10 to 5 in movie table.

```
UPDATE
    movie
SET
    rating = rating/2;
```

Expressions in HAVING Clause

Example: Get all the genres with an average profit of at least 100 crores.

```
SELECT
    genre
FROM
```

```
    movie
GROUP BY
    genre
HAVING
    AVG(collection_in_cr - budget_in_cr) >= 100;
```

Output:

genre
Action
Animation
Mystery
...

SQL Functions

SQL provides many built-in functions to perform various operations over data that is stored in tables. SQL functions can be divided into different categories such as:

1. String functions
2. Date functions
3. Numeric functions
4. Cast functions

MySQL String Functions

String functions in SQL are used to manipulate and operate on string values or character data.

Function	Description	Example
`CONCAT(s1, s2, ...)`	Joins strings together	`CONCAT('Hello', ' ', 'World')` → `'Hello World'`
`UPPER(s)`	Converts to uppercase	`UPPER('hello')` → `'HELLO'`
`LOWER(s)`	Converts to lowercase	`LOWER('HELLO')` → `'hello'`
`LENGTH(s)`	Returns string length in characters	`LENGTH('MySQL')` → `5`

Example 1: Get movie names in uppercase.

```
SELECT
    UPPER(name) AS movie_name_upper
FROM
    movie;
```

Example 2: Find movies whose name contains `'avengers'` (case-insensitive).

```
SELECT
    name
FROM
    movie
WHERE
    UPPER(name) LIKE UPPER('%avengers%');
```

Output:

name
Avengers: Age of Ultron
Avengers: Endgame

Example 3: Get the length of each movie title.

```
SELECT
    name,
    LENGTH(name) AS title_length
FROM
    movie
ORDER BY
    title_length DESC;
```

Note: `UPPER()` and `LOWER()` enable case-insensitive searches. `LENGTH()` returns the number of characters.

MySQL Date Functions

Date functions extract and format date/time values.

Function	Description	Example
`DATE_FORMAT(date, format)`	Formats a date as a string	`DATE_FORMAT('2024-03-15', '%Y-%m')` → `'2024-03'`
`DATEDIFF(date1, date2)`	Days between two dates	`DATEDIFF('2024-12-31', '2024-01-01')` → `365`
`NOW()`	Current date and time	`NOW()` → `'2024-06-14 10:30:00'`
`CURDATE()`	Current date	`CURDATE()` → `'2024-06-14'`
`YEAR(date)`	Extract year	`YEAR('2010-07-14')` → `2010`
`MONTH(date)`	Extract month	`MONTH('2010-07-14')` → `7`

Common DATE_FORMAT specifiers:

Specifier	Output	Example
`%Y`	4-digit year	2024

Specifier	Output	Example
`%m`	Month (01-12)	03
`%d`	Day (01-31)	15
`%H`	Hour (00-23)	14
`%i`	Minutes (00-59)	30

Example 1: Get the movie title and release year.

```
SELECT
    name,
    YEAR(release_date) AS release_year
FROM
    movie;
```

Output:

name	release_year
The Matrix	1999
Inception	2010
The Dark Knight	2008
...	...

Example 2: Find how many movies were released in each month of 2010.

```
SELECT
    DATE_FORMAT(release_date, '%m') AS month,
    COUNT(*) AS total_movies
FROM
    movie
WHERE
    YEAR(release_date) = 2010
GROUP BY
    month;
```

Example 3: Get movies released more than 10 years ago.

```
SELECT
    name,
    release_date,
    DATEDIFF(CURDATE(), release_date) AS days_since_release
FROM
    movie
WHERE
    DATEDIFF(CURDATE(), release_date) > 3650;
```

CAST Function

In database management systems, the `CAST` function is used to convert a value from one data type to another data type.

Syntax: `CAST(value AS data_type)`

Example: `CAST(YEAR(release_date) AS UNSIGNED)`

The CAST function takes:

1. **Value:** The value that you want to convert into a specific data type.
2. **Data type:** The data type to which you want to convert the value.

Example: Find how many movies were released in each month of the year 2010.

```
SELECT
    DATE_FORMAT(release_date, '%m') AS month,
    COUNT(*) AS total_movies
FROM
    movie
WHERE
    YEAR(release_date) = 2010
GROUP BY
    month;
```

Arithmetic Functions

Arithmetic functions in SQL are used to perform mathematical operations on numeric values. Some commonly used arithmetic functions are `FLOOR`, `CEIL`, `ROUND`.

FLOOR function: Rounds a number to the nearest integer, below its current value.

- Syntax: `FLOOR(number)`
- Example: `SELECT FLOOR(2.3);` → Output: 2

CEIL function: Rounds a number to the nearest integer above its current value.

- Syntax: `CEIL(number)`
- Example: `SELECT CEIL(-2.7);` → Output: -2

ROUND function: Rounds a number to a specified number of decimal places.

- Syntax: `ROUND(number, decimal_places)`
- Example: `SELECT ROUND(2.345, 2);` → Output: 2.35
- Example: `SELECT ROUND(2.345, 1);` → Output: 2.3

String Functions (Reference)

The `UPPER()` / `LOWER()` examples above cover case-insensitive search. See **MySQL String Functions** for `CONCAT()` and `LENGTH()`.

SQL Set Operations

The SQL set operation is used to combine two or more SQL queries.

Sets for understanding:

- Cast in 'Sherlock Holmes' movie: Robert D Jr., Jude Law, Mark Strong

- Cast in 'Avengers Endgame' movie: Robert D Jr., Chris Evans, Mark Ruffalo

Operator	Description	Result
INTERSECT	Actors who acted in both movies	Robert D Jr.
MINUS	Actors in Sherlock Holmes but not Avengers Endgame	Jude Law, Mark Strong
UNION	Unique actors in either movie	Jude Law, Mark Strong, Robert D Jr., Chris Evans, Mark Ruffalo
UNION ALL	Doesn't eliminate duplicate results	Jude Law, Mark Strong, Robert D Jr., Robert D Jr., Chris Evans, Mark Ruffalo

Syntax:

```
SELECT
    C1, C2
FROM
    table_name_1
SET_OPERATOR
SELECT
    C1, C2
FROM
    table_name_2;
```

Basic rules when combining two SQL queries using set operators:

1. Each SELECT statement must have the same number of columns.
2. The columns must have similar data types.
3. The columns in each SELECT statement must be in the same order.

Note: `MINUS` is used in Oracle. PostgreSQL and SQLite use `EXCEPT` for the same operation. Example: `SELECT ... EXCEPT SELECT ...`

Example: Get ids of actors who acted in both Sherlock Holmes (id = 6) and Avengers Endgame (id = 15).

```
SELECT
    actor_id
FROM
    cast
WHERE
    movie_id = 6
INTERSECT
SELECT
    actor_id
FROM
    cast
WHERE
    movie_id = 15;
```

Output: actor_id = 6

ORDER BY Clause in Set Operations

- ORDER BY clause can appear only once at the end of the query containing multiple SELECT statements.
- While using set operators, individual SELECT statements cannot have ORDER BY clause. Sorting can be done based on the columns that appear in the first SELECT query. It is recommended to sort using column positions.

Example:

```
SELECT actor_id FROM cast WHERE movie_id = 6
UNION
SELECT actor_id FROM cast WHERE movie_id = 15
ORDER BY 1 DESC;
```

Pagination in Set Operations

Similar to ORDER BY clause, LIMIT and OFFSET clauses are used at the end of the list of queries.

Example: Get the first 5 ids of actors who acted in Sherlock Holmes (id=6) or Avengers Endgame (id=15). Sort ids in the descending order.

```
SELECT actor_id FROM cast WHERE movie_id = 6
UNION
SELECT actor_id FROM cast WHERE movie_id = 15
ORDER BY 1 DESC
LIMIT 5;
```